

Acquisition et Nettoyage des Données

Master 1 Data & IA — 2025-2026

Ahmad ALMAKSOUR — Faculté de Gestion, Économie et Sciences

Évaluation en CC (50%) + Examen final (50%)

Objectifs de la séance

01 Comprendre ce qu'est un pipeline de données et sa place dans l'architecture data

02 Distinguer ETL et ELT et savoir choisir selon le contexte

03 Identifier et comparer les principales sources de données

04 Maîtriser les formats de fichiers : CSV, JSON, XML, Parquet

05 Prendre en main la stack Python data science et son écosystème

Plan de la séance

01

Pipelines
& architecture

02

ETL vs ELT

03

Les 5V
du Big Data

04

Sources de
données

05

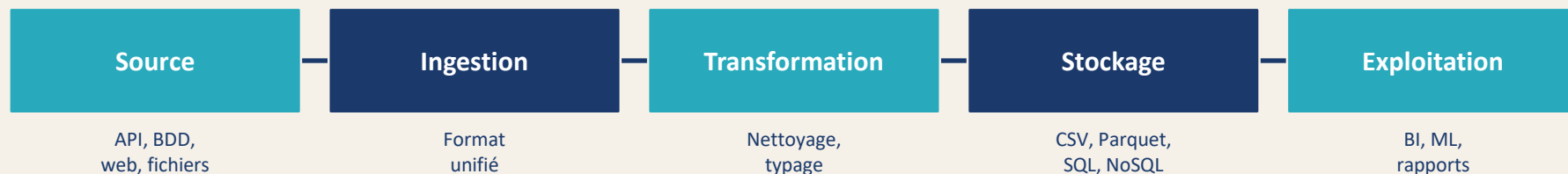
Formats
de fichiers

06

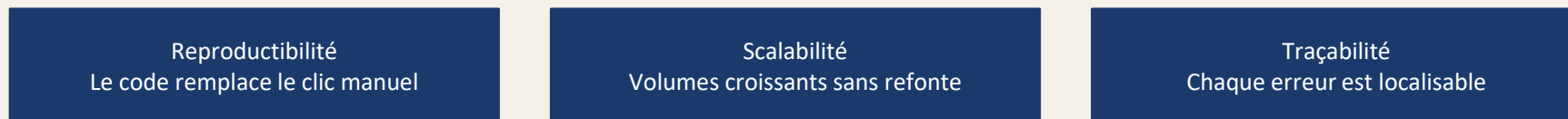
Stack Python
& qualité

Qu'est-ce qu'un pipeline de données ?

Un pipeline de données est une séquence automatisée d'étapes permettant de collecter, transformer et stocker des données pour les rendre exploitables par des équipes data ou des modèles d'IA.



Les 3 garanties d'un pipeline bien conçu



Des pipelines partout - exemples du monde réel

Netflix

Chaque visionnage génère un événement. Le pipeline collecte 700 milliards d'événements/jour, les agrège et alimente les algorithmes de recommandation.

INSEE

La DSN collecte les fiches de paie de 1,6 million d'entreprises, les transforme en statistiques d'emploi et de salaires publiées chaque trimestre.

Banque

Les transactions carte bancaire sont ingérées en temps réel, enrichies de données de fraude et analysées pour bloquer les paiements suspects en moins de 300 ms.

Data analyst

Vous téléchargez le CSV DVF, le chargez dans Pandas, nettoyez les valeurs manquantes et produisez un rapport. Ce processus, lui aussi, est un pipeline.

Deux modes d'exécution : batch et temps réel

Traitement par lots (Batch)

- Les données sont accumulées puis traitées ensemble
- Fréquence : horaire, quotidienne, mensuelle
- Simplicité de mise en œuvre et de débogage
- Cas d'usage : rapports mensuels, statistiques INSEE, DVF
- Outils : Python + cron, Apache Airflow, Spark Batch

Traitement en temps réel (Streaming)

- Chaque événement est traité dès son arrivée
- Latence : de la milliseconde à quelques secondes
- Complexité plus élevée, infrastructure plus coûteuse
- Cas d'usage : détection de fraude, recommandations live
- Outils : Apache Kafka, Spark Streaming

Dans ce cours, on travaillera principalement en mode batch - c'est le point de départ de 90 % des projets data.

ETL — Extract, Transform, Load



Caractéristiques clés

- La transformation a lieu **AVANT** le chargement et les données arrivent propres dans l'entrepôt
- Serveur ETL dédié qui porte toute la logique de transformation
- Adapté aux données structurées, schéma stable, volumes maîtrisés
- Limite : le serveur ETL devient un goulot d'étranglement dès que les volumes augmentent

Outils typiques

- Talend
- Informatica PowerCenter
- Stambia

ETL en pratique - exemple Python simplifié

Un ETL minimal, c'est trois fonctions Python :

```
import pandas as pd

def extract(path):
    """E - Lecture depuis la source brute"""
    return pd.read_csv(path)

def transform(df):
    """T - Nettoyage et typage"""
    df = df.dropna(subset=['price', 'number_of_reviews'])
    df['price'] = df['price'].str.replace('$',
    '').str.replace(',', '').astype(float)
    df['prix_par_nuit'] = df['price']
    return df[df['prix_par_nuit'].between(10, 1000)]

def load(df, dest):
    """L - Stockage dans l'entrepôt"""
    df.to_parquet(dest, compression='snappy')

# Pipeline complet
raw = extract('airbnb_paris.csv')
clean = transform(raw)
load(clean, 'airbnb_clean.parquet')
```

Ce qu'on retient

Séparation claire

Chaque étape fait une seule chose => facile à déboguer

Reproductible

Relancer = même résultat à chaque fois

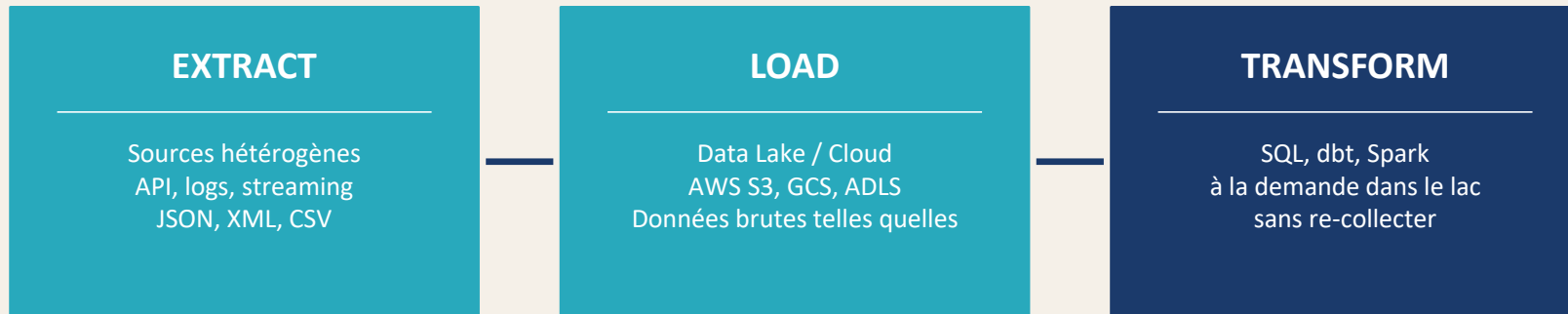
Testable

On peut tester `extract()` ou `transform()` indépendamment

Évolutif

Ajouter une source = nouvelle fonction `extract()`

ELT — Extract, Load, Transform



Caractéristiques clés

- Les données brutes sont stockées en premier ensuite on transforme à la demande
- Toute la puissance de calcul du cloud est disponible pour la transformation (BigQuery, Snowflake)
- Flexible : si le besoin métier change, on re-transforme sans re-collecter
- Adapté aux grands volumes, données variées, schéma susceptible d'évoluer

Outils typiques

- dbt (data build tool)
- Airbyte / Fivetran
- Apache Spark
- BigQuery / Snowflake
- Databricks

ETL vs ELT — Comparaison

Critère	ETL	ELT
Ordre	Extract → Transform → Load	Extract → Load → Transform
Lieu de transformation	Serveur ETL dédié	Dans le Data Warehouse / Lake
Schéma	Schema-on-write (fixe à l'écriture)	Schema-on-read (flexible à la lecture)
Infrastructure	On-premise ou hybride	Cloud-native principalement
Volumétrie	Petits à moyens volumes	Grands volumes, Big Data
Flexibilité	Faible => refonte coûteuse	Élevée => re-transformation possible
Re-traitement	Rejeu complet nécessaire	Sans re-collecter la source

ETL ou ELT — Aide à la décision

Choisissez ETL si...

- ✓ Données sensibles => contrôle avant stockage obligatoire
- ✓ Système legacy ou DWH on-premise déjà en place
- ✓ Volume maîtrisé, schéma stable dans le temps
- ✓ Conformité RGPD avec minimisation à la source requise

Choisissez ELT si...

- ✓ Architecture cloud moderne (AWS, GCP, Azure)
- ✓ Données variées : structurées et non-structurées
- ✓ Volumes importants et croissants, besoin d'élasticité
- ✓ Exploration fréquente, schéma susceptible de changer

Les 5V du Big Data

Volume

Quantité de données
générées et stockées
Combien ?

Vélocité

Vitesse de génération
et de traitement
À quelle vitesse ?

Variété

Structurées, semi-struct.
non-structurées, images
Sous quelle forme ?

Véracité

Qualité, fiabilité
et exactitude
Peut-on faire confiance ?

Valeur

Utilité métier
et ROI de l'exploitation
À quoi ça sert ?

→ Ces 5 dimensions guident vos choix d'architecture, d'outils et de format de stockage.

Les 5V - Application concrète : les données

DVF

Les Demandes de Valeurs Foncières (DVF) sont le registre officiel de toutes les transactions immobilières en France. Analysons-les à travers les 5V.

V	DVF Paris 2023
Volume	~3 millions de transactions par an sur toute la France. Fichier Paris 2023 : ~70 000 lignes, 30 colonnes.
Vélocité	Mise à jour annuelle par le gouvernement. Pas de temps réel - un traitement batch annuel suffit.
Variété	Colonnes numériques (prix, surface), textuelles (adresse, type de local), dates - mais toutes structurées.
Véracité	Source officielle notariale avec obligation légale de déclaration. Très fiable - quelques erreurs de saisie malgré tout.
Valeur	Permet d'analyser l'évolution des marchés, détecter des inégalités territoriales, alimenter des modèles d'estimation immobilière.

Sources de données (1/4) — Open Data et bases de données

Open Data

- data.gouv.fr — DVF, INSEE, Santé, Éducation, Budget
- Eurostat — statistiques européennes harmonisées
- Kaggle / UCI ML Repository — datasets académiques
- Accès libre, citable, reproductible
- Mise à jour parfois lente, qualité variable selon la source

Bases de données

SQL

PostgreSQL, MySQL, SQLite
Schéma rigide, transactions ACID

NoSQL

MongoDB (documents), Redis (clé-valeur)
Cassandra (colonnes), Neo4j (graphes)

DWH

Snowflake, BigQuery, Redshift
Optimisé pour les lectures analytiques

Sources de données (2/4) — API REST

Mode d'accès préférentiel en data engineering

Comment ça fonctionne



```
import requests

# Appel simple à l'API Sirene de l'INSEE
url = 'https://api.insee.fr/entreprises/sirene/V3/siren/552032534'
headers = {'Authorization': 'Bearer <votre_token>'}

reponse = requests.get(url, headers=headers)
data = reponse.json() # dictionnaire Python
print(data['uniteLegale']['denominationUniteLegale'])
```

Pourquoi c'est préférable au fichier plat

- Données toujours à jour et le producteur gère les mises à jour
- Pas de fichier local à gérer ni à versionner
- Reproductibilité : le même code donne le même résultat
- Documentation standardisée
- Gestion des droits d'accès intégrée (token, OAuth)

Sources de données (3/3) — Web Scraping

Principe et outils

- Extraction du contenu HTML d'une page web via son DOM
- BeautifulSoup , parsing d'une page statique
- Scrapy — crawler pour parcourir des centaines de pages
- Selenium / Playwright , sites qui utilisent JavaScript
- Cas d'usage : prix en ligne, offres d'emploi, actualités

Contraintes légales et éthiques

robots.txt

Fichier indiquant les pages interdites aux bots , toujours vérifier avant de scraper

CGU

Les conditions générales peuvent interdire la collecte automatisée

RGPD

Directive CNIL 2020 : données personnelles protégées même publiquement accessibles

Rate limiting

Respecter un délai entre les requêtes, ne pas surcharger le serveur cible

L'API reste toujours préférable au scraping quand elle existe, le scraping est un dernier recours, pas un premier choix.

Formats de fichiers - CSV

Le format le plus universel... et le plus limité

```
# Exemple de fichier DVF en CSV (séparateur |)
date_mutation|valeur_fonciere|type_local|surface_reelle_bati
2023-03-15|350000|Appartement|52
2023-05-22|1200000|Maison|185

# Lecture avec Pandas
df = pd.read_csv('dvf.csv', sep='|', low_memory=False)
```

Limites

- ✗ Pas de types intégrés => `pd.read_csv()` doit tout inférer
- ✗ Séparateurs variables (virgule, point-virgule, pipe)
- ✗ Lent à lire sur de gros volumes (scan ligne par ligne)
- ✗ Taille non compressée => souvent 5 à 10× plus gros que Parquet

Forces

- ✓ Lisible par n'importe quel outil (Excel, R, Pandas, SQL)
- ✓ Facile à inspecter avec un éditeur de texte
- ✓ Aucune dépendance => format universel
- ✓ Idéal pour partager des petits datasets

Forces

- ✗ Impossible de lire une seule colonne sans tout charger
- ✗ Encodages parfois incohérents (UTF-8, latin-1, cp1252)
- ✗ Aucune notion de schéma => facile de corrompre sans s'en rendre compte

Formats de fichiers - JSON et XML

JSON — JavaScript Object Notation

```
{
  "date_mutation": "2023-03-15",
  "valeur_fonciere": 350000,
  "type_local": "Appartement",
  "adresse": {
    "rue": "10 rue Lepic",
    "code_postal": "75018"
  }
}
```

- Format natif des API REST, vous le rencontrerez partout
- Semi-structuré : supporte les objets imbriqués
- Verbeux , clés répétées pour chaque ligne
- Pandas : `pd.read_json()` ou `pd.json_normalize()`

XML — eXtensible Markup Language

```
<mutation>
  <date>2023-03-15</date>
  <valeur>350000</valeur>
  <type>Appartement</type>
  <adresse>
    <rue>10 rue Lepic</rue>
  </adresse>
</mutation>
```

- Standard d'échange historique (services publics, banques)
- Validable via un schéma XSD , robuste mais lourd
- Très verbeux , les balises doublent la taille des données
- Progressivement remplacé par JSON dans les nouveaux projets

Formats de fichiers - Apache Parquet

Le format de référence pour l'analytique en production

- Stockage en colonnes (vs lignes pour CSV) → lectures sélectives ultra-rapides
- Compression intégrée (Snappy, ZSTD) → 5 à 20 fois plus petit qu'un CSV équivalent
- Schéma intégré → les types sont conservés, pas de ré-inférence à la lecture
- Prédicats pushdown → filtrer les données directement sur le disque sans charger en mémoire
- Interopérable : Pandas, Spark, DuckDB, BigQuery, Snowflake lisent tous Parquet

```
import pandas as pd

# Écrire en Parquet
df.to_parquet('dvf_clean.parquet',
              compression='snappy')

# Lire tout ou en partie
df = pd.read_parquet('dvf_clean.parquet')
df2 = pd.read_parquet('dvf_clean.parquet',
                      columns=['valeur_fonciere', 'surface_reelle_bati'])
```

Format	Taille	Lecture	Écriture
CSV	350 Mo	12.4 s	8.2 s
CSV + gzip	48 Mo	14.1 s	19.5 s
Parquet (Snappy)	22 Mo	0.8 s	2.1 s
Parquet (Zstd)	14 Mo	0.9 s	2.8 s

DuckDB interroge des fichiers Parquet directement via SQL, sans les charger en mémoire — c'est pourquoi Parquet est le format pivot des architectures data modernes.

Formats de fichiers - Comment choisir ?

CSV

- ✓ Universel, lisible, Excel l'ouvre
- ✗ Pas de types, lent sur volumes
- Partage ponctuel, petit dataset < 100 Mo, interopérabilité maximale

JSON

- ✓ Natif pour les API, imbriqué
- ✗ Verbeux, requêtes SQL difficiles
- Réponse d'API, données semi-structurées, config et paramètres

XML

- ✓ Validable (XSD), standard legacy
- ✗ Très verbeux, parsing lent
- Interfaces avec des systèmes bancaires ou gouvernementaux anciens

Parquet

- ✓ Colonne, compressé, typé
- ✗ Non lisible humain, nécessite pyarrow
- Pipeline de production, dataset > 100 Mo, analytique performante

Qualité des données - pourquoi c'est fondamental

Un modèle d'IA entraîné sur des données corrompues produit des prédictions corrompues. La qualité des données est la première responsabilité du data engineer.

Complétude	Conformité	Cohérence	Unicité	Précision
				
Toutes les valeurs attendues sont présentes	Les valeurs respectent le format et le type attendus	Pas de contradictions entre colonnes ou entre sources	Pas de doublons ; chaque entité est représentée une seule fois	Les valeurs reflètent fidèlement la réalité

Qualité des données - Valeurs manquantes

Détecter les valeurs manquantes avec Pandas

```
# Compter les manquants par colonne
df.isna().sum()

# Taux de manquants en pourcentage
(df.isna().sum() / len(df) * 100).sort_values(ascending=False)

# Supprimer les lignes sans prix ni surface
df_clean = df.dropna(subset=['valeur_fonciere',
'surface_reelle_bati'])

# Remplacer les manquants par la médiane
df['surface_reelle_bati'] = df['surface_reelle_bati'].fillna(
    df['surface_reelle_bati'].median()
)
```

Règle importante

Avant d'imputer, demandez-vous pourquoi la valeur manque. Une surface manquante sur un garage n'a pas le même sens que sur un appartement.

Que faire selon le taux de manquants ?

< 5 %

Supprimer les lignes ou imputer par médiane / mode

5 – 30 %

Imputer intelligemment selon le contexte métier

> 30 %

Remettre en question la colonne , la supprimer ou recollecte

Qualité des données - Doublons et valeurs aberrantes

Détecter et traiter les doublons

```
# Compter les doublons
df.duplicated().sum()

# Doublons sur une clé métier
df.duplicated(subset=['id_mutation',
                     'date_mutation', 'adresse_nom_voie']).sum()

# Supprimer (garder le premier)
df_clean = df.drop_duplicates(
    subset=['id_mutation', 'date_mutation'],
    keep='first'
)
```

```
# Méthode IQR
Q1 = df['valeur_fonciere'].quantile(0.25)
Q3 = df['valeur_fonciere'].quantile(0.75)
IQR = Q3 - Q1

bf = df[
    (df['valeur_fonciere'] >= Q1 - 1.5*IQR) &
    (df['valeur_fonciere'] <= Q3 + 1.5*IQR)
]
```

Ce que révèlent les anomalies

- ! Une valeur à 0 € pour un appartement parisien est presque toujours une erreur de saisie
- ! Un prix au m² de 200 000 € peut être une erreur de virgule ou une erreur d'unité
- ! Avant de supprimer une aberration, vérifiez le type de local et l'adresse - certaines sont légitimes

La stack Python data science - Vue d'ensemble

NumPy

Calcul numérique
tableaux N-dim

Pandas

Manipulation
DataFrame / ETL léger

Matplotlib Seaborn

Visualisation
graphiques stats

Requests BeautifulSoup

Scraping
& API HTTP

NumPy — La brique de base du calcul numérique

NumPy (Numerical Python) est la bibliothèque fondamentale de l'écosystème Python data science. Pandas, Scikit-learn et TensorFlow reposent tous dessus.

```
import numpy as np

# Créer un tableau
arr = np.array([1, 2, 3, 4, 5])
print(arr.shape)    # (5,)
print(arr.dtype)    # int64

# Opérations vectorisées — PAS de boucle Python
arr * 2              # [2, 4, 6, 8, 10]
arr ** 2             # [1, 4, 9, 16, 25]
np.sqrt(arr)        # [1., 1.41, 1.73, 2., 2.23]

# Tableau 2D (matrice)
mat = np.array([[1,2,3],[4,5,6]])
print(mat.shape)    # (2, 3) = 2 lignes, 3 colonnes
mat[0, :]           # première ligne : [1, 2, 3]
```

Pourquoi NumPy est si rapide

Tableaux homogènes

Tous les éléments ont le même type → stockage contigu en mémoire

Code C en dessous

Les opérations sont exécutées en C compilé, pas en Python interprété

Vectorisation

Une opération sur le tableau entier en une seule instruction CPU

En pratique, vous utiliserez NumPy indirectement via Pandas. Mais comprendre NumPy explique pourquoi Pandas est rapide - et pourquoi les boucles Python sur les DataFrames sont à éviter.

Pandas - L'outil central de la manipulation de données

Pandas transforme NumPy en outil orienté données : colonnes nommées, index, types hétérogènes et méthodes SQL-like.

```
import pandas as pd

# Charger et inspecter
df = pd.read_csv('dvf.csv', sep='|')
df.shape      # (70425, 32)
df.dtypes     # types de chaque colonne
df.head(3)    # premières lignes
df.describe() # stats descriptives

# Sélection et filtrage
df['valeur_fonciere']      # une colonne (Series)
df[['type_local', 'valeur_fonciere']] # plusieurs colonnes
df[df['type_local'] == 'Appartement'] # filtrage booléen

# Agrégation (équivalent SQL GROUP BY)
df.groupby('type_local')['valeur_fonciere'].median()
```

Structure d'un DataFrame

Index

Étiquettes des lignes (0, 1, 2... ou dates)

Columns

Noms des colonnes - chacune est une Series

dtypes

Un type par colonne (int64, float64, object, datetime64)

NaN

Valeur spéciale pour les données manquantes

Pandas - Les opérations que vous utiliserez chaque jour

Lecture / Écriture

```
pd.read_csv('f.csv')  
pd.read_parquet('f.parquet')  
df.to_csv('out.csv')  
df.to_parquet('out.parquet')
```

Exploration

```
df.shape df.dtypes df.info()  
df.head(5) df.describe() df.sample(10)
```

Nettoyage

```
df.dropna() df.fillna(val)  
df.drop_duplicates()  
df['col'] = pd.to_datetime(df['col'])
```

Sélection

```
df['col'] df[['c1','c2']] df.loc[0:5,  
'col']  
df[df['col'] > 100] df.query('col >  
100')
```

Agrégation

```
df.groupby('col').agg({'val':  
['mean','median','count']})  
df.pivot_table(values='val', index='col',  
aggfunc='sum')
```

Jointure

```
pd.merge(df1, df2, on='cle') # inner  
join  
pd.concat([df1, df2], axis=0) # empiler
```

ydata-profiling - Générer un rapport qualité en 3 lignes

ydata-profiling est un outil industriel qui analyse automatiquement chaque colonne d'un DataFrame et génère un rapport HTML interactif.

```
from ydata_profiling import ProfileReport

# Générer le rapport
profile = ProfileReport(
    df,
    title='DVF Paris 2023 - Rapport Qualité',
    explorative=True
)

# Exporter en HTML
profile.to_file('rapport.html')

# Afficher dans Jupyter
profile.to_notebook_iframe()
```

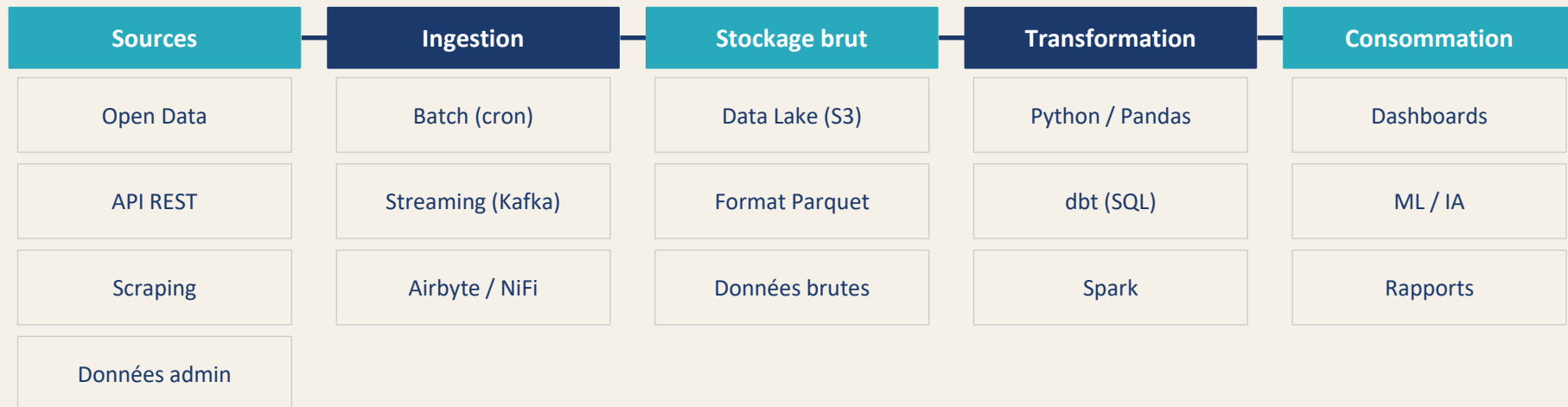
Ce que le rapport contient

Overview	Dimensions, types, alertes qualité
Variables	Distribution de chaque colonne
Correlations	Matrice de corrélation interactive
Missing values	Heatmap des données manquantes
Sample	Premières et dernières lignes

Sur un grand dataset, le profiling peut prendre plusieurs minutes. Travaillez sur `df.sample(10_000)` en phase d'exploration. Pour les > 1M lignes : `ProfileReport(df, minimal=True)`.

Architecture data moderne - Vue d'ensemble

Comment s'articulent tous les concepts vus aujourd'hui dans une architecture réelle ?



ETL $\leftrightarrow$ ELT selon si la transformation précède ou suit le chargement dans le stockage brut

Bonnes pratiques du data engineer

01

Versionner le code

Tout script de traitement va dans Git. Le notebook qui tourne une fois puis disparaît n'existe pas.

02

Documenter les transformations

Chaque étape de nettoyage doit être justifiée dans le code. Pourquoi avoir filtré ces lignes ?

03

Ne jamais modifier la source

Les données brutes sont sacrées. Toujours travailler sur une copie ou dans une couche transformation.

04

Automatiser les contrôles

Les tests de qualité (Great Expectations) s'exécutent à chaque ingestion, pas une fois lors du développement.

05

Tracer les métadonnées

Conserver : date de collecte, source, version, nombre de lignes. Indispensable pour auditer et déboguer.

06

Choisir le bon format dès le départ

Passer de CSV à Parquet après coup sur 10 To coûte cher. La décision architecture se prend en amont.

Récapitulatif - Ce qu'on a vu aujourd'hui

01

Pipelines de données

Séquence automatisée Extract → Transform → Load. Batch ou temps réel selon le besoin.

04

Sources de données

Open data, BDD SQL/NoSQL, API REST, scraping, données administratives — chacune a ses contraintes.

02

ETL vs ELT

ETL transforme avant de charger (on-premise). ELT charge d'abord puis transforme dans le cloud.

05

Formats de fichiers

CSV (simplicité), JSON/XML (échanges API), Parquet (performance analytique en production).

03

Les 5V

Volume, Vitesse, Variété, Véracité, Valeur - grille d'analyse universelle pour tout projet data.

06

Qualité & stack Python

Valeurs manquantes, doublons, aberrations. NumPy + Pandas + ydata-profiling pour les traiter.

Du cours au TP — Ce que vous allez faire maintenant

Le TP de cette séance vous demande d'appliquer directement les concepts du cours sur un dataset réel.

Concept du cours	Ce que vous ferez dans le notebook
Pipeline	Vous implémentez un pipeline complet extract → transform (nettoyage) → load (Parquet)
ETL ou ELT ?	Vous devrez identifier à quoi correspond votre travail et justifier votre réponse
5V du Big Data	Vous caractérisez le dataset DVF selon les 5 dimensions vues en cours
CSV vs Parquet	Vous mesurez vous-mêmes la différence de taille et de vitesse de lecture
Sources	Vous identifiez la catégorie de source à laquelle appartient DVF
Qualité Pandas	Manquants, doublons, aberrations — détectés et traités sans outil automatisé

Rendu attendu : TP1_DVF_NOM_Prenom.ipynb - toutes les cellules exécutées, cellules Markdown rédigées

Questions ?

ahmad.almosaalmaksour@lacatholille.fr